Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Corso di Artificial and Swarm Intelligence

Relazione Progetto

E.L.B.E.R.R.

Efficient Logistics by Exploration with Robotic Retrieval

Autore: Daniele Barbagallo

Matricola: 1000015334

Docente: Prof. Mario Francesco Pavone

Anno Accademico 2025/2026

Indice

1 Ambiente e Simulazione **3**
1.1 Griglia . . . 3
1.2 Magazzini . . . 3
1.3 Ciclo di simulazione . . . 4
1.4 Gestione delle collisioni . . . 4
1.5 Prenotazione degli ingressi ai magazzini . . . 5
1.6 Comunicazione . . . 5

2 Agenti **6**
2.1 Attributi . . . 6
2.2 Percezione (Sense) . . . 6
2.3 Macchina a stati . . . 6
2.4 Pathfinding . . . 7
2.5 Consegna . . . 7
2.5.1 Strategie di ricarica . . . 7

3 Strategie di Esplorazione **8**
3.1 Funzioni condivise della classe base . . . 8
3.2 Smart Random Walk . . . 9
3.3 Frontier . . . 9
3.4 Greedy . . . 10
3.5 Sector . . . 10
3.6 Repulsion . . . 10

4 Interfaccia Utente **11**
4.1 Barra laterale . . . 11
4.2 Tab Simulazione . . . 11
4.3 Tab Benchmark . . . 12

5 Risultati Sperimentali **13**
5.1 Mappa A . . . 13
5.1.1 Test esaustivo — 32.768 simulazioni . . . 13
5.1.2 Test randomizzato — 100×3 seed . . . 15
5.2 Mappa B . . . 17
5.2.1 Test esaustivo — 32.768 simulazioni . . . 17
5.2.2 Test randomizzato — 100×3 seed . . . 19
5.3 Considerazioni sui risultati . . . 21
5.3.1 Confronto tra le due mappe . . . 21
5.3.2 Impatto della strategia dominante . . . 21

5.3.3 Team eterogenei vs omogenei 22
5.3.4 Impatto del raggio di visibilità 22
5.3.5 Robustezza ai seed casuali 23
5.3.6 Riepilogo 23
5.3.7 Riepilogo delle euristiche utilizzate 24

Capitolo 1

Ambiente e Simulazione

E.L.B.E.R.R. (*Efficient Logistics by Exploration with Robotic Retrieval*) è una simulazione multi-agente di recupero e consegna oggetti in un magazzino a pianta variabile. Cinque agenti autonomi operano in parallelo su una griglia 2D con visione e comunicazione locali, senza alcun coordinamento centralizzato.

1.1 Griglia

L'ambiente è rappresentato da una griglia quadrata $N \times N$ le cui celle appartengono a uno dei seguenti tipi:

- `EMPTY` (0) — corridoio libero, percorribile dagli agenti;
- `WALL` (1) — ostacolo o scaffale, non percorribile;
- `WAREHOUSE` (2) — cella interna al magazzino (zona di deposito/ricarica);
- `ENTRANCE` (3) — porta d'ingresso del magazzino;
- `EXIT` (4) — porta d'uscita del magazzino.

Gli agenti possono attraversare solo celle `EMPTY`, `ENTRANCE` e `EXIT`; le porte dei magazzini sono state gestite in modo che siano percorribili esclusivamente come destinazione finale di un percorso, mai come transito intermedio. Questo perchè è stato osservato che in alcuni casi, gli agenti consideravano erroneamente le porte come scorciatoie dovute alla presenza di troppi agenti nelle vicinanze, causando ulteriore congestione e collisioni evitabili.

1.2 Magazzini

Ogni magazzino è una zona rettangolare con un ingresso e un'uscita separati. L'istanza viene caricata da un file JSON che specifica la pianta, le posizioni dei magazzini e la distribuzione casuale degli oggetti. Gli agenti consegnano gli oggetti entrando dall'ingresso, depositandoli su una cella interna e uscendo dall'uscita.

1.3 Ciclo di simulazione

Ad ogni tick il simulatore esegue, in ordine:

1. **Sense** — ogni agente aggiorna la propria mappa locale tramite ray casting nel raggio di visione v_r;
2. **Communicate** — gli agenti entro il raggio di comunicazione reciproco c_r si scambiano mappe locali, oggetti noti e posizioni degli agenti conosciuti (merge bidirezionale);
3. **Decide** — ogni agente sceglie la prossima mossa in base alla propria strategia di esplorazione;
4. **Act** — le mosse vengono applicate con gestione delle collisioni;
5. **Pickup/Deliver** — gli agenti raccolgono oggetti e li consegnano;
6. **Metriche** — aggiornamento dei contatori.

La simulazione termina quando tutti gli oggetti sono stati consegnati oppure si raggiunge il limite di 500-750 tick.

1.4 Gestione delle collisioni

La prevenzione delle collisioni è implementata con un approccio *ibrido distribuito*: ogni agente tenta attivamente di evitarle basandosi su informazioni locali, mentre il simulatore funge da arbitro imparziale per i conflitti inevitabili.

Autogestione locale

Ad ogni tick, nel costruire la propria destinazione, ogni agente filtra le celle attraverso la funzione di pathfinding utilizzando l'insieme delle posizioni occupate (`known_agents`) entro una finestra di validità temporale di 5 tick. Agenti che comunicano direttamente si scambiano inoltre posizioni note, propagando informazioni sullo stato del sistema via *gossip protocol*.

Questo meccanismo permette ai singoli agenti di *evitare proattivamente* collisioni di cui sono a conoscenza, manifestando una forma di coordinamento localizzato e biologicamente realistica.

Arbitrato centrale

Tuttavia, in un sistema decentralizzato con comunicazione locale limitata, non è possibile garantire che ogni agente conosca la posizione in tempo reale di tutti gli altri. Conflitti emergono inevitabilmente quando:

- Due agenti che non si vedono e non hanno comunicato tentano la stessa destinazione;
- L'informazione su un agente è scaduta (oltre 5 tick).

In questi casi, il simulatore applica una risoluzione deterministica *prima di eseguire alcun movimento*, garantendo che non si producano mai stati incoerenti:

1. Un agente che non si muove (`dest=None`) *blocca* la propria cella: nessun altro può entrarvi.

2. Se due o più agenti richiedono la stessa cella, vince chi trasporta un oggetto (priorità funzionale); a parità di stato, vince l'ID minore (arbitrio deterministico per garantire no-deadlock). Gli altri restano fermi e potranno riprovare al prossimo tick.

3. Le porte (`ENTRANCE` e `EXIT`) sono automaticamente aggiunte all'insieme protected durante la pianificazione, evitando che gli agenti le attraversino innecessariamente.

1.5 Prenotazione degli ingressi ai magazzini

Quando un agente trasporta un oggetto, deve scegliere a quale ingresso consegnarlo. La selezione non è statica: ad ogni tick viene rivalutata tramite un costo dinamico (descritto nel Capitolo 3) che combina distanza, congestione locale e prenotazioni note.

Per evitare oscillazioni, la scelta viene vincolata con un *lock di 8 tick*: l'agente cambia destinazione solo se trova un ingresso con costo almeno il 20% inferiore a quello attuale. La prenotazione viene condivisa via gossip durante la comunicazione, così gli altri agenti possono tenerne conto nel calcolo della congestione.

1.6 Comunicazione

Due agenti comunicano se la distanza di Manhattan tra le loro posizioni è $\leq \min(c_{r,a}, c_{r,b})$. Lo scambio è bidirezionale e include:

- Mappa locale (merge delle celle esplorate);
- Insieme degli oggetti noti non ancora raccolti;
- Posizioni note degli altri agenti (si conserva la più recente);
- Prenotazioni di consegna ai magazzini (anti-congestione).

Non esiste un canale broadcast globale: la conoscenza si propaga solo per contatto diretto, creando un effetto di *gossip protocol* distribuito.

Capitolo 2

Agenti

Ogni agente è un'entità autonoma caratterizzata da posizione, batteria, mappa locale, insieme di oggetti noti, stato interno e strategia di esplorazione. Tutti gli agenti condividono la stessa classe base; il comportamento specifico è determinato dalla strategia iniettata alla creazione.

2.1 Attributi

- **Batteria** — 500 unità iniziali, decrementata di 1 per ogni spostamento. A esaurimento l'agente entra nello stato `DEPLETED` e si ferma.
- **Raggio di visione** v_r — configurabile tra 1 e 3 celle (distanza di Manhattan).
- **Raggio di comunicazione** c_r — configurabile tra 1 e 3 celle.
- **Mappa locale** — dizionario $(r, c) \to$ `CellType` che accumula le osservazioni dirette e le informazioni ricevute dagli altri agenti.
- **Oggetti noti** — insieme delle posizioni di oggetti rilevati ma non ancora raccolti, aggiornato dalla visione diretta e dalla comunicazione.
- **Posizioni agenti noti** — per ogni agente conosciuto si conserva la posizione e il tick di osservazione più recente.

2.2 Percezione (Sense)

Ad ogni tick, l'agente calcola le celle visibili entro il raggio v_r tramite *ray casting* basato sull'algoritmo di Bresenham: una cella è visibile solo se il segmento rettilineo che la collega all'agente non attraversa muri. Le celle visibili aggiornano la mappa locale e il set `seen_cells`; gli oggetti nel campo visivo vengono aggiunti a `known_objects`. Se una cella precedentemente contenente un oggetto risulta vuota, l'oggetto viene rimosso dai noti (pulizia stale).

2.3 Macchina a stati

Il ciclo decisionale è governato dai seguenti stati:

1. **EXPLORING** — nessun oggetto noto; la strategia di esplorazione guida il movimento.

2. **MOVING_TO_OBJECT** — un oggetto è stato individuato; l'agente si dirige verso di esso tramite A*.
3. **CARRYING** — l'agente ha un oggetto raccolto e si dirige verso il magazzino più vicino noto.
4. **DELIVERING** — l'agente è entrato nel magazzino e raggiunge la cella interna di deposito.
5. **EXITING_WAREHOUSE** — dopo la consegna, l'agente esce dalla porta di uscita.
6. **DEPLETED** — batteria esaurita, agente fermo.

2.4 Pathfinding

Il pathfinding utilizza A* con euristica Manhattan sulla griglia. Le celle occupate da altri agenti vengono passate come set di celle bloccate per evitare collisioni. La variante `allow_warehouse` abilita il transito attraverso le celle interne del magazzino durante le fasi di consegna e uscita.

2.5 Consegna

La sequenza di consegna avviene in tre fasi:

1. L'agente raggiunge la cella `ENTRANCE` del magazzino scelto;
2. Attraversa l'interno e deposita l'oggetto su una cella `WAREHOUSE`;
3. Esce dalla cella `EXIT` e torna in stato `EXPLORING`.

La selezione dell'ingresso di consegna utilizza un costo dinamico che combina distanza e congestione, con un meccanismo di prenotazione soft per evitare oscillazioni (descritto nella Sezione dedicata alla classe base delle strategie).

2.5.1 Strategie di ricarica

La batteria non viene mai ricaricata: ogni agente dispone di 500 unità totali, che devono bastare per l'intera simulazione. La motivazione per cui non sono state introdotte strategie di ricarica consiste nel fatto che siano ritenute non necessarie al raggiungimento dell'obiettivo sperimentale: nelle istanze testate gli agenti esaurivano l'energia solo in casi estremamente rari. Introdurre routine di ricarica avrebbe comportato costi temporali (tick persi per la ricarica) e avrebbe potuto peggiorare l'efficienza complessiva, rendendo inoltre più difficili confronti diretti tra strategie. Per questi motivi si é preferito mantenere un budget fisso di passi, ponendo l'accento sull'efficienza di movimento e sulla qualità delle strategie di esplorazione.

Capitolo 3

Strategie di Esplorazione

Le cinque strategie implementano l'interfaccia `ExplorationStrategy`; ognuna definisce il metodo `next_move` che restituisce la prossima cella verso cui spostarsi. Tutte condividono una **priorità universale**: se l'agente trasporta un oggetto, la destinazione è il magazzino più vicino; se conosce un oggetto non reclamato, si dirige verso di esso. Solo quando nessuna delle due condizioni è attiva la strategia di esplorazione vera e propria viene invocata.

3.1 Funzioni condivise della classe base

La classe `ExplorationStrategy` espone sette metodi ereditati da tutte le strategie.

Priority-Move. Implementa la priorità universale. Se l'agente trasporta un oggetto, lo indirizza all'ingresso del magazzino scelto; altrimenti, se conosce oggetti non ancora reclamati, lo dirige verso il più vicino (Manhattan). Solo se nessuna condizione è attiva il controllo passa alla logica di esplorazione specifica.

Selezione ingresso di consegna. Sceglie dinamicamente l'ingresso di consegna combinando distanza e congestione. Il costo per un ingresso e è:

$$C(e) \;=\; d(\text{agent}, e) \;+\; 4{,}5 \cdot Q_{\text{local}} \;+\; 1{,}5 \cdot R_{\text{reserved}} \;+\; \varepsilon_{\text{tie}}$$

dove Q_{local} è il numero di agenti noti entro raggio 2 dall'ingresso, R_{reserved} è il numero di prenotazioni attive su quell'ingresso, e ε_{tie} è un tie-break deterministico basato sull'ID dell'agente. L'ingresso scelto viene *prenotato* con un lock di 8 tick; un cambio avviene solo se il nuovo costo è almeno il 20% inferiore a quello corrente, riducendo le oscillazioni.

Find-Frontier. Il metodo `ExplorationStrategy._find_frontiers` individua le frontiere di copertura: celle `EMPTY` già osservate dall'agente che confinano con almeno una cella `EMPTY` non ancora vista nella mappa locale. Per ogni cella osservata si verificano i quattro vicini cardinali; se almeno uno di essi è una cella percorribile non presente in `seen_cells`, la cella viene marcata come frontiera. Il risultato viene memorizzato in cache per `agent_id` e ricalcolato solo quando aumenta il numero di celle in `seen_cells`, riducendo il costo computazionale nei cicli ripetuti di esplorazione.

Nota implementativa: questo metodo era inizialmente stato sviluppato per testare strategie con visione esclusivamente locale. Tuttavia, per mantenere omogeneita tra tutti gli agenti e facilitare il confronto equo delle strategie, e stata successivamente scartata a favore di un approccio unificato dove tutte le strategie accedono alla mappa globale. Il metodo rimane nel codice come utility riutilizzabile per futuri sviluppi.

Coverage targets. Fornisce i target di esplorazione utilizzando la conoscenza globale della mappa (accedendo direttamente a `env.grid.empty_cells()`), con tre livelli di priorità:

1. Celle `EMPTY` mai osservate localmente (*unexplored*);
2. Celle `EMPTY` osservate ma non visitate da almeno $\max(8, N/2)$ tick (*stale*);
3. Tutte le celle `EMPTY` (*patrolling*).

Il passaggio al livello successivo avviene solo quando il precedente è vuoto. Questa scelta di design consente alle strategie di operare con consapevolezza della struttura complessiva del magazzino, migliorando l'efficienza esplorativa pur mantenendo autonomia locale nelle decisioni di movimento.

Guadagno informativo (*information gain*). Euristica condivisa da piú strategie (Smart Random Walk e Greedy). Per una cella candidata t il guadagno informativo è il numero di celle non ancora viste che l'agente osserverebbe se si trovasse in t:

$$\text{info_gain}(t) = \left| \{(r,c) : |r - t_r| + |c - t_c| \leq v_r \ \wedge \ (r,c) \notin \text{seen_cells}\} \right|$$

dove v_r è il raggio di visibilità dell'agente. Valori alti indicano zone dense di celle inesplorate e guidano l'agente verso le aree più informative.

3.2 Smart Random Walk

Passeggiata casuale intelligente. Per ogni vicino percorribile viene calcolato un punteggio che bilancia:

- **Guadagno informativo** — celle ignote nell'intorno del candidato, pesato da $w_{\text{info}} = 1{,}2$;
- **Stale bonus** — tick dall'ultima visita alla cella, pesato da $w_{\text{stale}} = 0{,}8$;
- **Separazione** — distanza media dagli agenti noti, pesata da $w_{\text{sep}} = 0{,}5$;
- **Penalità rivisitazione** — penalità se la cella è stata visitata di recente, pesata da $w_{\text{revisit}} = 1{,}0$;
- **Rumore casuale** — componente $\mathcal{U}(0,\ 0{,}3)$ per evitare pattern deterministici.

La mossa viene selezionata con probabilità proporzionale al punteggio (*softmax*), mantenendo una componente stocastica che impedisce cicli.

3.3 Frontier

Esplorazione frontier-based classica. L'agente identifica le frontiere: celle percorribili adiacenti ad almeno una cella `UNKNOWN` nella propria mappa locale. Tra tutte le frontiere, seleziona quella più vicina per distanza di Manhattan e vi si dirige tramite A*. Quando non esistono più frontiere (mappa completamente esplorata), l'agente esegue un random walk tra i vicini liberi.

3.4 Greedy

Seleziona il target di esplorazione che minimizza un costo composito:

$$\text{cost}(t) = d(\text{agent}, t) + d(t, \text{wh}_{\text{nearest}}) - \text{info_gain}(t)$$

dove d è la distanza di Manhattan, $\text{wh}_{\text{nearest}}$ è il magazzino noto più vicino e $\text{info_gain}(t)$ conta le celle non ancora osservate entro il raggio di visibilità v_r del target t (cfr. formula nella Sezione 3.1). L'effetto è un bias verso zone inesplorate e vicine ai magazzini. In assenza di target utili, l'agente si avvicina al peer più vicino per favorire lo scambio di mappa.

3.5 Sector

La griglia viene suddivisa in bande orizzontali, una per agente (in base a `agent_id` mod N). Ogni agente copre sistematicamente la propria banda con un pattern a serpentina, poi si sposta sulle bande adiacenti non coperte. Questo garantisce copertura distribuita e uniforme senza bisogno di comunicazione esplicita.

3.6 Repulsion

Tra tutti i coverage target disponibili, seleziona quello che massimizza il punteggio:

$$\text{score}(t) = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} d(t, a) - d(\text{agent}, t)$$

dove $\mathcal{A}$ è l'insieme delle posizioni degli agenti noti. La strategia massimizza la distanza media dagli altri agenti (repulsione) e minimizza il costo di spostamento, producendo una dispersione uniforme dello sciame sulle zone inesplorate.

Capitolo 4

Interfaccia Utente

L'intero sistema è controllabile tramite una dashboard web realizzata con **Streamlit**, avviabile con il comando `streamlit run app.py`. L'interfaccia è organizzata in una barra laterale di configurazione globale e due tab principali: *Simulazione* e *Benchmark*.

4.1 Barra laterale

La sidebar consente di selezionare il **file istanza** (mappa JSON) e il **seed** di riproducibilità (valore -1 per seed casuale). Questi parametri sono condivisi da entrambe le modalità.

4.2 Tab Simulazione

Il pannello di simulazione è suddiviso in tre colonne:

- **Configurazione** (sinistra) — permette di impostare il numero di agenti (1–10), il limite di tick (100–750) e il delay tra frame. Per ogni agente si sceglie la strategia di esplorazione, il raggio di visione ($v_r \in \{1, 2, 3\}$) e il raggio di comunicazione ($c_r \in \{1, 2\}$). È possibile caricare un preset JSON salvato in precedenza o personalizzare le icone degli agenti e dei pacchi.

- **Visualizzazione** (centro) — mostra un rendering in tempo reale della griglia, generato frame per frame tramite Pygame in modalità off-screen. Ogni cella viene colorata in base al tipo (muro, corridoio, magazzino, ingresso, uscita); gli agenti sono cerchi colorati con etichetta ID; i pacchi e i magazzini hanno icone dedicate. Una barra di avanzamento indica il tick corrente e lo stato della simulazione.

- **Stato** (destra) — riporta in tempo reale il tick attuale, il conteggio degli oggetti consegnati e la batteria residua di ogni agente.

Al termine della simulazione vengono mostrati i risultati finali: oggetti consegnati, tasso di completamento, tick totali, energia media e tempo CPU. Una tabella riassuntiva per agente elenca strategia, raggio, passi effettuati e batteria finale. Un grafico mostra la curva cumulativa delle consegne nel tempo.

La simulazione può essere ripetuta più volte senza ricaricare la pagina; uno **storico** delle run precedenti consente di confrontare le configurazioni testate nella stessa sessione. Al termine è possibile scaricare il preset corrente in formato JSON per riutilizzarlo.

4.3 Tab Benchmark

La modalità benchmark consente di eseguire automaticamente un numero arbitrario di simulazioni con configurazioni diverse, per esplorare lo spazio dei parametri.

La configurazione avviene tramite:

- **Strategie**: selezione multipla tra le strategie disponibili oppure strategia singola fissa;
- **Raggi**: range di visione e comunicazione, ciascuno in modalità casuale (intervallo) o fissa (valore singolo);
- **Numero di preset**: il sistema calcola e mostra lo spazio di ricerca totale (combinazioni per agente elevate al numero di agenti) e permette di scegliere quanti preset testare.

I preset vengono generati senza duplicati (campionamento casuale oppure enumerazione esaustiva se il numero richiesto copre l'intero spazio). Durante l'esecuzione una barra di avanzamento mostra il preset corrente e la percentuale completata.

Al termine vengono prodotti:

- Una **tabella ordinabile** con tutti i risultati (tasso di completamento, tick, energia, configurazione, strategia dominante);
- Un **grafico a barre** dei tick di completamento, colorato per strategia dominante, con linea della media;
- Un **grafico a barre** degli oggetti consegnati per preset;
- Un file **ZIP esportabile** contenente: `summary.csv`, `results.json` (configurazioni dettagliate), `curves.json` (curve di consegna), `metadata.json` (parametri della sessione) e le immagini dei grafici in formato PNG.

Questo workflow ha permesso di generare i dati presentati nel Capitolo 5.

Capitolo 5

Risultati Sperimentali

I benchmark sono stati condotti su due mappe (**Mappa A** e **Mappa B**), ciascuna con due batterie di test:

1. **Test esaustivo (32.768 simulazioni)** — tutte le combinazioni di 4 strategie (Frontier, Greedy, Sector, Repulsion), raggi di visione $v_r \in \{2, 3\}$ e raggio di comunicazione fisso $c_r = 2$, per un team di 5 agenti. Limite: 500 tick.

2. **Test randomizzato (100×3 seed)** — 100 preset casuali con 5 strategie (incluso Smart Random Walk), $v_r \in \{1, 2, 3\}$, $c_r \in \{1, 2\}$, ripetuti su tre seed indipendenti (42, 43, 44) per valutare la robustezza statistica. Limite: 500 tick.

Nota sulla configurazione del test esaustivo. Smart Random Walk non è stato incluso nel test esaustivo (32.768 simulazioni) poiché la sua aggiunta come quinta strategia avrebbe moltiplicato il numero di combinazioni in modo esponenziale, rendendo il benchmark computazionalmente infatibile. Abbinato ai 2 raggi di visione e al raggio di comunicazione fisso, il costo temporale totale sarebbe stato inaccettabile. Per questo motivo, è stata scelta Random Walk come sacrificio dalla configurazione esaustiva. Nonostante la sua componente stocastica, Random Walk dimostrava efficienza comparabile alle altre strategie; tuttavia, era meno versatile nei contesti eterogenei, rendendone la scelta più accettabile. Smart Random Walk rimane incluso nel test randomizzato dove la ricchezza dei dati permette comunque di valutarne il comportamento.

5.1 Mappa A

5.1.1 Test esaustivo — 32.768 simulazioni

Sono state eseguite 32.768 simulazioni in circa 3 h 15 min di tempo CPU.

- **Tasso di completamento**: 96,4 % dei preset ha consegnato tutti e 10 gli oggetti (31.602 su 32.768). Il tasso medio di consegna è $\overline{r} = 0{,}990$.

- **Tick medi di completamento**: $\overline{T} = 162{,}9$ tick (calcolati solo sui preset completati al 100 %). Il miglior preset ha completato in soli 77 tick.

- **Energia media**: $\overline{E} = 173{,}1$ unità consumate per agente.

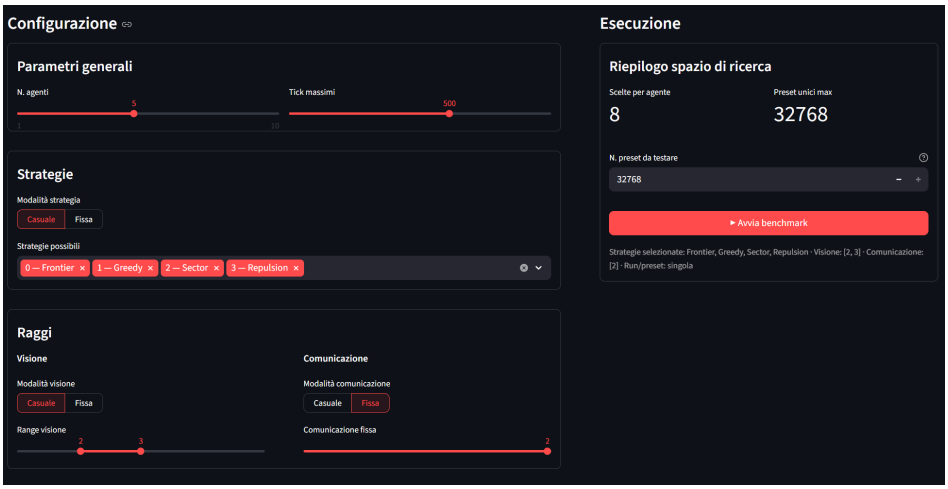

Figura 5.1: Configurazione della Mappa A utilizzata nel test esaustivo.

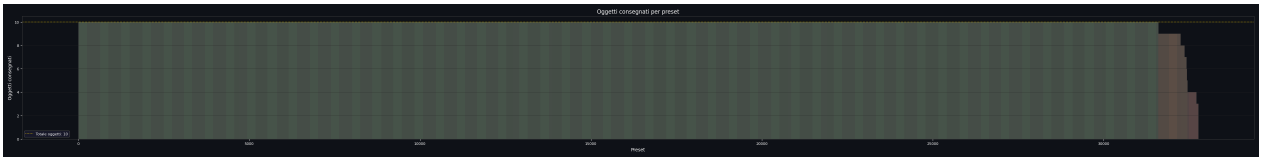

Figura 5.2: Mappa A — 32k sim: distribuzione degli oggetti consegnati per preset.

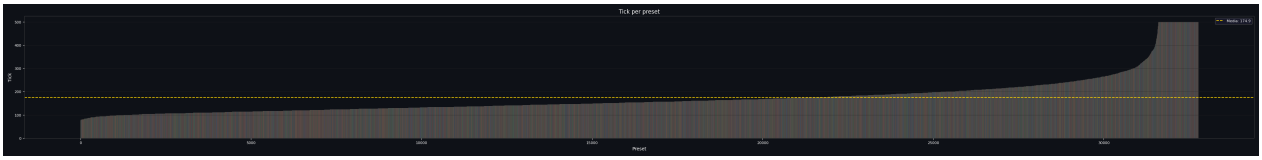

Figura 5.3: Mappa A — 32k sim: tick di completamento per preset.

Top 10 preset

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 23781	1×Greedy + 1×Repulsion + 3×Sector	A1:Sect(v3,c2) A2:Repu(v2,c2) A3:Gree(v3,c2) A4:Sect(v2,c2) A5:Sect(v2,c2)	100.0%	77	75.8
2	Preset 23783	1×Greedy + 2×Repulsion + 2×Sector	A1:Sect(v3,c2) A2:Repu(v2,c2) A3:Gree(v3,c2) A4:Sect(v2,c2) A5:Repu(v2,c2)	100.0%	77	75.8
3	Preset 3298	2×Frontier + 1×Greedy + 1×Repulsion + 1×Sector	A1:Fron(v2,c2) A2:Repu(v2,c2) A3:Gree(v3,c2) A4:Sect(v2,c2) A5:Fron(v3,c2)	100.0%	79	77
4	Preset 24802	2×Frontier + 1×Greedy + 1×Repulsion + 1×Sector	A1:Repu(v2,c2) A2:Fron(v2,c2) A3:Gree(v3,c2) A4:Sect(v2,c2) A5:Fron(v3,c2)	100.0%	79	77
5	Preset 26146	2×Frontier + 1×Greedy + 1×Repulsion + 1×Sector	A1:Repu(v2,c2) A2:Gree(v3,c2) A3:Fron(v2,c2) A4:Sect(v2,c2) A5:Fron(v3,c2)	100.0%	79	77
6	Preset 9698	1×Frontier + 2×Greedy + 1×Repulsion + 1×Sector	A1:Gree(v2,c2) A2:Gree(v2,c2) A3:Repu(v3,c2) A4:Sect(v2,c2) A5:Fron(v3,c2)	100.0%	79	77.4
7	Preset 11554	1×Frontier + 1×Greedy + 1×Repulsion + 2×Sector	A1:Gree(v2,c2) A2:Repu(v2,c2) A3:Sect(v2,c2) A4:Sect(v2,c2) A5:Fron(v3,c2)	100.0%	79	77.4
8	Preset 11558	1×Greedy + 1×Repulsion + 3×Sector	A1:Gree(v2,c2) A2:Repu(v2,c2) A3:Sect(v2,c2) A4:Sect(v2,c2) A5:Sect(v3,c2)	100.0%	79	77.4
9	Preset 11938	1×Frontier + 2×Greedy + 1×Repulsion + 1×Sector	A1:Gree(v2,c2) A2:Repu(v3,c2) A3:Gree(v2,c2) A4:Sect(v2,c2) A5:Fron(v3,c2)	100.0%	79	77.4
10	Preset 12066	1×Frontier + 1×Greedy + 1×Repulsion + 2×Sector	A1:Gree(v2,c2) A2:Repu(v3,c2) A3:Sect(v2,c2) A4:Sect(v2,c2) A5:Fron(v3,c2)	100.0%	79	77.4

Figura 5.4: Mappa A — 32k sim: classifica dei 10 migliori preset per tick di completamento.

Tabella 5.1: Mappa A — Top 10 preset nel test esaustivo (32k simulazioni).

#	Preset	Composizione team	Tick	$\overline{E}$
1	23781	Sect(v_3) Repu(v_2) Gree(v_3) Sect(v_2) Sect(v_2)	77	75,8
2	23783	Sect(v_3) Repu(v_2) Gree(v_3) Sect(v_2) Repu(v_2)	77	75,8
3	3298	Fron(v_2) Repu(v_2) Gree(v_3) Sect(v_2) Fron(v_3)	79	77,0
4	24802	Repu(v_2) Fron(v_2) Gree(v_3) Sect(v_2) Fron(v_3)	79	77,0
5	26146	Repu(v_2) Gree(v_3) Fron(v_2) Sect(v_2) Fron(v_3)	79	77,0
6	9698	Gree(v_2) Gree(v_2) Repu(v_3) Sect(v_2) Fron(v_3)	79	77,4
7	11554	Gree(v_2) Repu(v_2) Sect(v_2) Sect(v_2) Fron(v_3)	79	77,4
8	11558	Gree(v_2) Repu(v_2) Sect(v_2) Sect(v_2) Sect(v_3)	79	77,4
9	11938	Gree(v_2) Repu(v_3) Gree(v_2) Sect(v_2) Fron(v_3)	79	77,4
10	12066	Gree(v_2) Repu(v_3) Sect(v_2) Sect(v_2) Fron(v_3)	79	77,4

Legenda: Fron = Frontier, Gree = Greedy, Sect = Sector, Repu = Repulsion. v_k = raggio di visibilità k. $c_r = 2$ per tutti.

5.1.2 Test randomizzato — 100×3 seed

La media complessiva sui tre seed è:

- **Tasso di completamento medio**: 99,6 % (96/100 completati per seed 42 e 44, 97/100 per seed 43).
- **Tick medi di completamento**: $\overline{T} = 158{,}5$ tick (media dei preset completati al 100 % sui tre seed).
- **Energia media**: $\overline{E} = 169{,}0$.

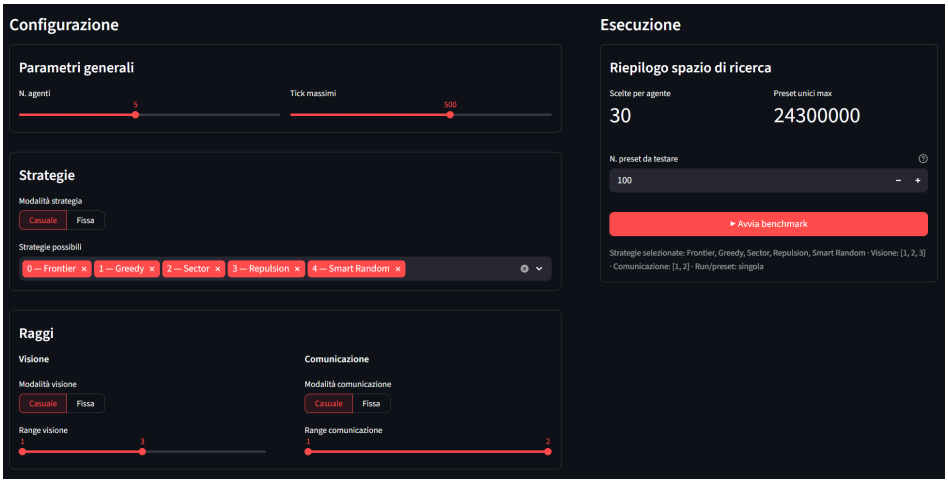

Figura 5.5: Configurazione della Mappa A utilizzata nel test randomizzato.

Seed 42

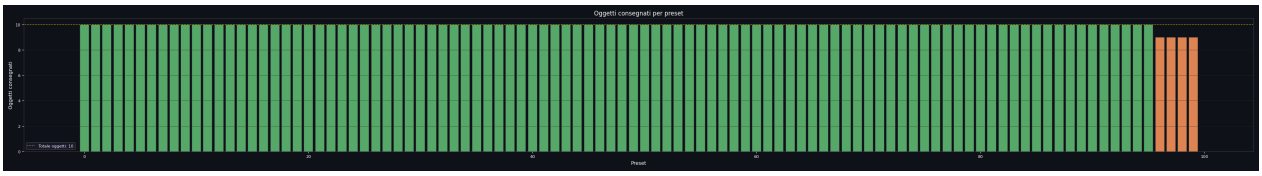

Figura 5.6: Mappa A — seed 42: oggetti consegnati per preset.

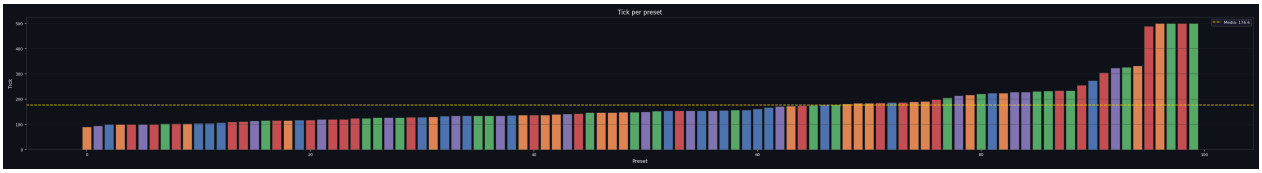

Figura 5.7: Mappa A — seed 42: tick di completamento per preset.

🏆 **Top 10 preset**

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 10	1×Frontier + 3×Greedy + 1×Smart Random	A1:Gree(v2,c1) A2:Fron(v3,c1) A3:Gree(v1,c1) A4:Smar(v1,c1) A5:Gree(v3,c1)	100.0%	90	89
2	Preset 9	1×Frontier + 1×Greedy + 1×Repulsion + 1×Sector + 1×Smart Random	A1:Smar(v1,c2) A2:Fron(v3,c2) A3:Repu(v3,c1) A4:Sect(v1,c2) A5:Gree(v3,c1)	100.0%	93	91.4
3	Preset 53	2×Frontier + 1×Repulsion + 1×Sector + 1×Smart Random	A1:Smar(v1,c1) A2:Fron(v3,c1) A3:Fron(v3,c1) A4:Repu(v2,c2) A5:Sect(v2,c2)	100.0%	99	97
4	Preset 16	1×Frontier + 2×Greedy + 2×Repulsion	A1:Gree(v3,c2) A2:Repu(v1,c1) A3:Gree(v1,c1) A4:Repu(v3,c1) A5:Fron(v2,c1)	100.0%	99	97.8
5	Preset 56	1×Greedy + 3×Repulsion + 1×Smart Random	A1:Smar(v1,c1) A2:Repu(v3,c2) A3:Repu(v3,c1) A4:Gree(v1,c1) A5:Repu(v3,c1)	100.0%	100	97.8
6	Preset 15	2×Repulsion + 3×Smart Random	A1:Smar(v2,c1) A2:Repu(v2,c1) A3:Repu(v1,c2) A4:Smar(v3,c1) A5:Smar(v2,c2)	100.0%	100	98.4
7	Preset 24	1×Greedy + 2×Repulsion + 1×Sector + 1×Smart Random	A1:Sect(v1,c1) A2:Gree(v2,c1) A3:Smar(v3,c2) A4:Repu(v2,c1) A5:Repu(v3,c1)	100.0%	100	99
8	Preset 58	1×Frontier + 3×Sector + 1×Smart Random	A1:Sect(v2,c1) A2:Smar(v3,c1) A3:Sect(v3,c1) A4:Sect(v2,c1) A5:Fron(v2,c2)	100.0%	102	99.8
9	Preset 51	2×Frontier + 2×Repulsion + 1×Smart Random	A1:Smar(v3,c1) A2:Repu(v1,c1) A3:Fron(v2,c1) A4:Fron(v2,c1) A5:Repu(v3,c2)	100.0%	102	100.2
10	Preset 72	1×Frontier + 2×Greedy + 2×Sector	A1:Gree(v1,c1) A2:Gree(v1,c1) A3:Sect(v3,c2) A4:Sect(v2,c1) A5:Fron(v3,c1)	100.0%	102	101.2

Figura 5.8: Mappa A — seed 42: classifica Top 10.

Seed 43

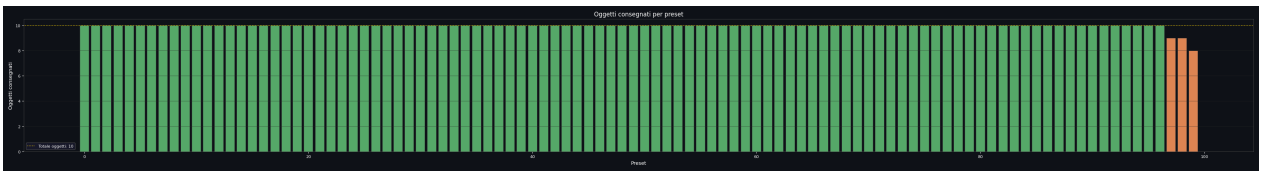

Figura 5.9: Mappa A — seed 43: oggetti consegnati per preset.

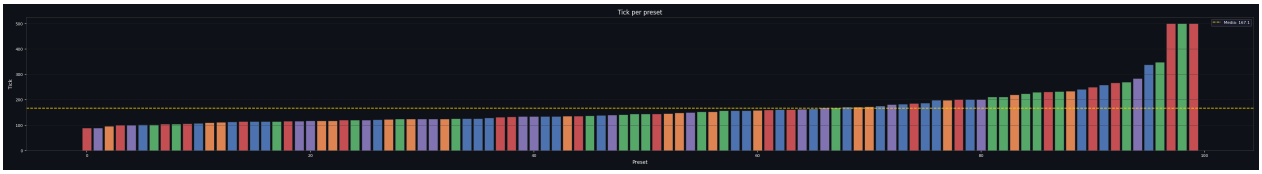

Figura 5.10: Mappa A — seed 43: tick di completamento per preset.

🏆 **Top 10 preset**

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 2	1×Greedy + 2×Repulsion + 1×Sector + 1×Smart Random	A1:Sect(v2,c1) A2:Gree(v3,c2) A3:Repu(v2,c2) A4:Repu(v3,c1) A5:Smar(v1,c2)	100.0%	88	86
2	Preset 16	2×Repulsion + 1×Sector + 2×Smart Random	A1:Sect(v2,c1) A2:Smar(v1,c2) A3:Repu(v2,c1) A4:Smar(v3,c1) A5:Repu(v3,c2)	100.0%	88	86.6
3	Preset 73	1×Frontier + 3×Greedy + 1×Smart Random	A1:Fron(v2,c2) A2:Smar(v1,c2) A3:Gree(v3,c1) A4:Gree(v3,c2) A5:Gree(v3,c1)	100.0%	95	93.2
4	Preset 90	2×Frontier + 3×Repulsion	A1:Repu(v2,c1) A2:Fron(v2,c2) A3:Fron(v1,c1) A4:Repu(v2,c2) A5:Repu(v3,c1)	100.0%	99	96.6
5	Preset 38	2×Greedy + 1×Sector + 2×Smart Random	A1:Smar(v3,c2) A2:Smar(v1,c2) A3:Gree(v3,c1) A4:Sect(v1,c1) A5:Gree(v3,c2)	100.0%	100	98.6
6	Preset 53	2×Frontier + 2×Repulsion + 1×Sector	A1:Fron(v3,c1) A2:Sect(v3,c2) A3:Repu(v1,c1) A4:Fron(v3,c1) A5:Repu(v3,c1)	100.0%	101	99.4
7	Preset 43	1×Frontier + 1×Greedy + 3×Sector	A1:Sect(v1,c1) A2:Gree(v1,c2) A3:Fron(v1,c2) A4:Sect(v2,c1) A5:Sect(v3,c1)	100.0%	101	100.2
8	Preset 40	1×Frontier + 1×Greedy + 2×Repulsion + 1×Smart Random	A1:Smar(v2,c1) A2:Gree(v1,c1) A3:Repu(v3,c1) A4:Repu(v3,c2) A5:Fron(v1,c2)	100.0%	104	101.8
9	Preset 4	1×Frontier + 1×Greedy + 1×Repulsion + 2×Sector	A1:Fron(v1,c1) A2:Sect(v3,c1) A3:Sect(v1,c2) A4:Repu(v1,c1) A5:Gree(v3,c2)	100.0%	104	103
10	Preset 64	1×Greedy + 3×Repulsion + 1×Smart Random	A1:Gree(v1,c1) A2:Repu(v1,c2) A3:Smar(v3,c2) A4:Repu(v3,c1) A5:Repu(v2,c1)	100.0%	105	102.6

Figura 5.11: Mappa A — seed 43: classifica Top 10.

Seed 44

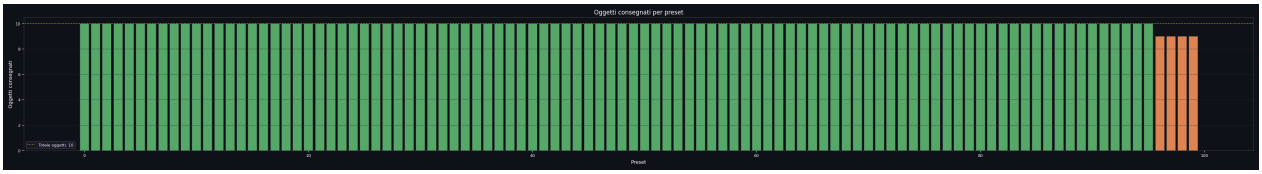

Figura 5.12: Mappa A — seed 44: oggetti consegnati per preset.

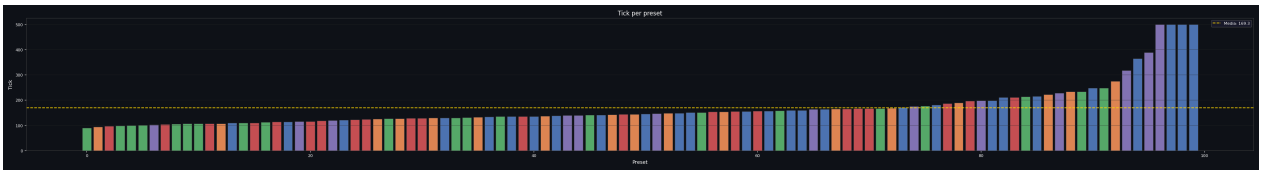

Figura 5.13: Mappa A — seed 44: tick di completamento per preset.

🏆 **Top 10 preset**

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 82	1×Frontier + 1×Greedy + 2×Sector + 1×Smart Random	A1:Smar(v3,c2) A2:Gree(v3,c1) A3:Sect(v2,c1) A4:Sect(v2,c2) A5:Fron(v1,c1)	100.0%	89	87.6
2	Preset 30	2×Frontier + 2×Greedy + 1×Sector	A1:Gree(v2,c1) A2:Fron(v2,c1) A3:Fron(v1,c1) A4:Sect(v1,c2) A5:Gree(v2,c1)	100.0%	93	91.4
3	Preset 50	2×Repulsion + 2×Sector + 1×Smart Random	A1:Repu(v3,c1) A2:Smar(v1,c2) A3:Sect(v1,c1) A4:Sect(v1,c1) A5:Repu(v1,c1)	100.0%	97	95
4	Preset 39	1×Frontier + 1×Repulsion + 2×Sector + 1×Smart Random	A1:Fron(v2,c1) A2:Smar(v3,c2) A3:Sect(v2,c1) A4:Sect(v1,c1) A5:Repu(v2,c1)	100.0%	98	94.8
5	Preset 72	4×Sector + 1×Smart Random	A1:Smar(v3,c1) A2:Sect(v2,c1) A3:Sect(v2,c2) A4:Sect(v1,c1) A5:Sect(v2,c2)	100.0%	99	96.6
6	Preset 92	1×Frontier + 1×Greedy + 1×Repulsion + 2×Sector	A1:Repu(v2,c1) A2:Sect(v2,c2) A3:Fron(v2,c2) A4:Sect(v3,c2) A5:Gree(v3,c1)	100.0%	101	99.8
7	Preset 34	1×Greedy + 1×Repulsion + 3×Smart Random	A1:Smar(v1,c1) A2:Smar(v2,c2) A3:Smar(v1,c2) A4:Repu(v1,c2) A5:Gree(v3,c1)	100.0%	102	100.6
8	Preset 14	1×Frontier + 2×Repulsion + 1×Sector + 1×Smart Random	A1:Smar(v2,c1) A2:Repu(v2,c2) A3:Sect(v1,c2) A4:Repu(v3,c1) A5:Fron(v1,c2)	100.0%	104	102.4
9	Preset 98	2×Repulsion + 3×Sector	A1:Repu(v1,c1) A2:Sect(v3,c2) A3:Sect(v1,c2) A4:Sect(v3,c2) A5:Repu(v3,c1)	100.0%	105	103.2
10	Preset 18	1×Frontier + 1×Greedy + 3×Sector	A1:Sect(v3,c2) A2:Fron(v3,c1) A3:Gree(v1,c2) A4:Sect(v2,c2) A5:Sect(v2,c1)	100.0%	107	105

Figura 5.14: Mappa A — seed 44: classifica Top 10.

5.2 Mappa B

5.2.1 Test esaustivo — 32.768 simulazioni

Tempo CPU totale: circa 2 h 30 min.

- **Tasso di completamento**: 99,6 % (32.652 su 32.768). $\bar{r} = 0{,}9995$.

- **Tick medi di completamento**: $\overline{T} = 105{,}7$ tick. Il miglior preset ha completato in 57 tick.
- **Energia media**: $\overline{E} = 104{,}7$.

Rispetto alla Mappa A, la Mappa B risulta sensibilmente più facile: tasso di completamento più alto (+3,2 pp), tick medi più bassi (−35 %) e minor consumo energetico (−40 %).

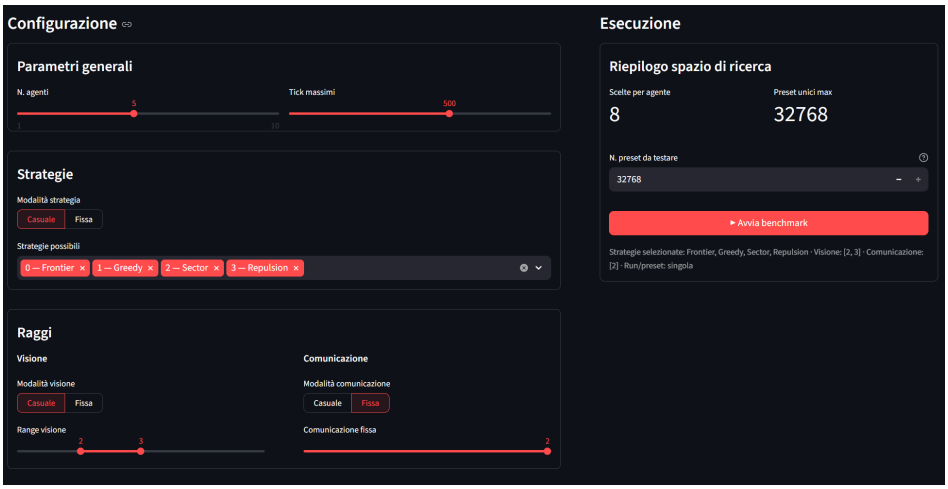

Figura 5.15: Configurazione della Mappa B utilizzata nel test esaustivo.

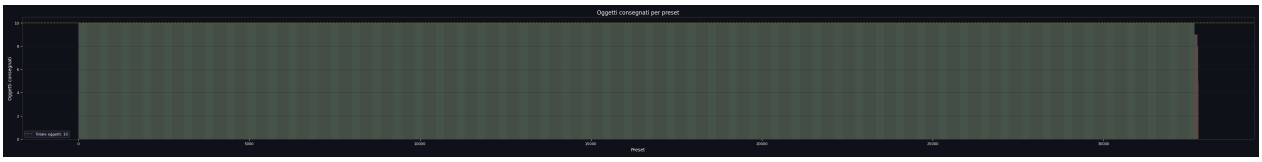

Figura 5.16: Mappa B — 32k sim: distribuzione degli oggetti consegnati per preset.

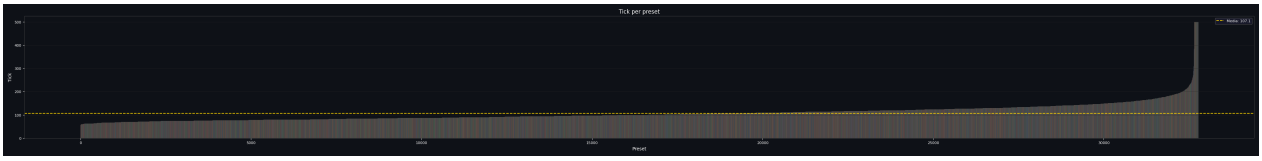

Figura 5.17: Mappa B — 32k sim: tick di completamento per preset.

Top 10 preset

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 31975	1×Greedy + 3×Repulsion + 1×Sector	A1:Repu(v3,c2) A2:Repu(v2,c2) A3:Gree(v3,c2) A4:Sect(v2,c2) A5:Repu(v2,c2)	100.0%	57	54.8
2	Preset 30119	1×Greedy + 3×Repulsion + 1×Sector	A1:Repu(v3,c2) A2:Gree(v2,c2) A3:Repu(v2,c2) A4:Sect(v2,c2) A5:Repu(v2,c2)	100.0%	57	55
3	Preset 27557	2×Repulsion + 3×Sector	A1:Repu(v2,c2) A2:Sect(v3,c2) A3:Repu(v2,c2) A4:Sect(v2,c2) A5:Sect(v2,c2)	100.0%	58	55.4
4	Preset 27759	1×Frontier + 3×Repulsion + 1×Sector	A1:Repu(v2,c2) A2:Repu(v2,c2) A3:Fron(v3,c2) A4:Sect(v3,c2) A5:Repu(v2,c2)	100.0%	58	55.6
5	Preset 24687	2×Frontier + 2×Repulsion + 1×Sector	A1:Repu(v2,c2) A2:Fron(v2,c2) A3:Fron(v3,c2) A4:Sect(v3,c2) A5:Repu(v2,c2)	100.0%	58	55.8
6	Preset 27754	2×Frontier + 2×Repulsion + 1×Sector	A1:Repu(v2,c2) A2:Repu(v2,c2) A3:Fron(v3,c2) A4:Sect(v3,c2) A5:Fron(v3,c2)	100.0%	58	55.8
7	Preset 27756	1×Frontier + 1×Greedy + 2×Repulsion + 1×Sector	A1:Repu(v2,c2) A2:Repu(v2,c2) A3:Fron(v3,c2) A4:Sect(v3,c2) A5:Gree(v3,c2)	100.0%	58	55.8
8	Preset 31657	1×Frontier + 2×Repulsion + 2×Sector	A1:Repu(v3,c2) A2:Sect(v3,c2) A3:Repu(v2,c2) A4:Sect(v3,c2) A5:Fron(v2,c2)	100.0%	58	55.8
9	Preset 31850	2×Frontier + 2×Repulsion + 1×Sector	A1:Repu(v3,c2) A2:Repu(v2,c2) A3:Fron(v3,c2) A4:Sect(v3,c2) A5:Fron(v3,c2)	100.0%	58	55.8
10	Preset 31852	1×Frontier + 1×Greedy + 2×Repulsion + 1×Sector	A1:Repu(v3,c2) A2:Repu(v2,c2) A3:Fron(v3,c2) A4:Sect(v3,c2) A5:Gree(v3,c2)	100.0%	58	55.8

Figura 5.18: Mappa B — 32k sim: classifica dei 10 migliori preset per tick di completamento.

Tabella 5.2: Mappa B — Top 10 preset nel test esaustivo (32k simulazioni).

#	Preset	Composizione team	Tick	$\overline{E}$
1	31975	Repu(v_3) Repu(v_2) Gree(v_3) Sect(v_2) Repu(v_2)	57	54,8
2	30119	Repu(v_3) Gree(v_2) Repu(v_2) Sect(v_2) Repu(v_2)	57	55,0
3	27557	Repu(v_2) Sect(v_3) Repu(v_2) Sect(v_2) Sect(v_2)	58	55,4
4	27759	Repu(v_2) Repu(v_2) Fron(v_3) Sect(v_3) Repu(v_2)	58	55,6
5	24687	Repu(v_2) Fron(v_2) Fron(v_3) Sect(v_3) Repu(v_2)	58	55,8
6	27754	Repu(v_2) Repu(v_2) Fron(v_3) Sect(v_3) Fron(v_3)	58	55,8
7	27756	Repu(v_2) Repu(v_2) Fron(v_3) Sect(v_3) Gree(v_3)	58	55,8
8	31657	Repu(v_3) Sect(v_3) Repu(v_2) Sect(v_3) Fron(v_2)	58	55,8
9	31850	Repu(v_3) Repu(v_2) Fron(v_3) Sect(v_3) Fron(v_3)	58	55,8
10	31852	Repu(v_3) Repu(v_2) Fron(v_3) Sect(v_3) Gree(v_3)	58	55,8

Legenda: Fron = Frontier, Gree = Greedy, Sect = Sector, Repu = Repulsion. v_k = raggio di visibilità k. $c_r = 2$ per tutti.

5.2.2 Test randomizzato — 100×3 seed

La media complessiva sui tre seed è:

- **Tasso di completamento medio**: 99,97 % (tutti i 100 preset completati con seed 42 e 43; 99/100 con seed 44).
- **Tick medi di completamento**: $\overline{T} = 112{,}6$ tick.
- **Energia media**: $\overline{E} = 111{,}6$.

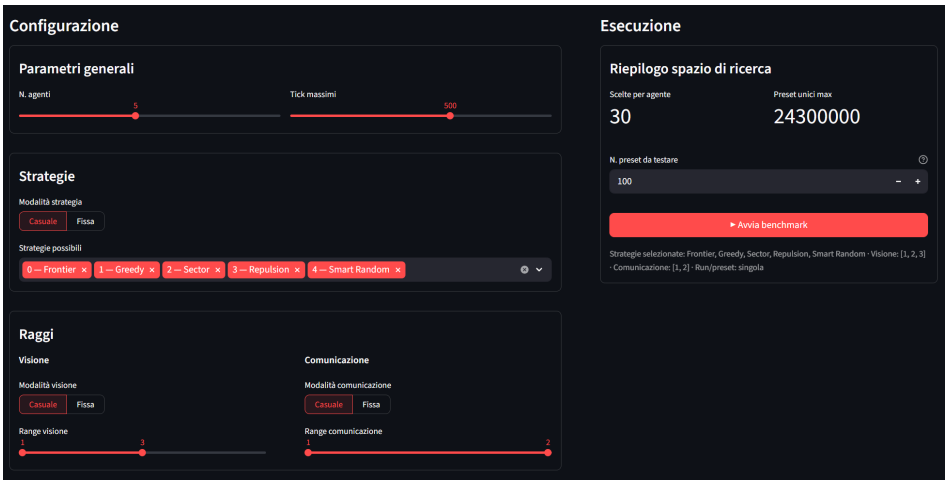

Figura 5.19: Configurazione della Mappa B utilizzata nel test randomizzato.

Seed 42

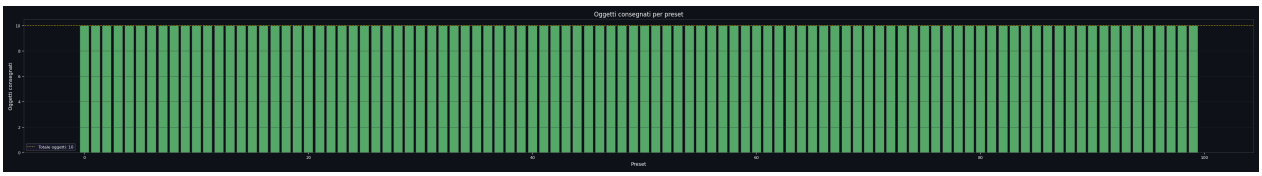

Figura 5.20: Mappa B — seed 42: oggetti consegnati per preset.

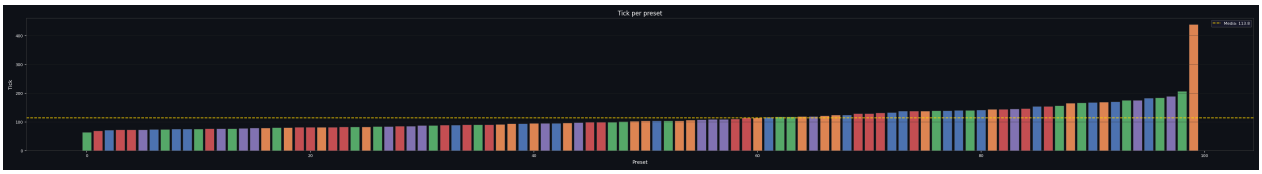

Figura 5.21: Mappa B — seed 42: tick di completamento per preset.

Top 10 preset

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 86	3×Sector + 2×Smart Random	A1:Sect(v2,c2) A2:Sect(v3,c1) A3:Smar(v3,c2) A4:Smar(v2,c2) A5:Sect(v3,c1)	100.0%	63	61.8
2	Preset 6	1×Greedy + 3×Repulsion + 1×Sector	A1:Sect(v3,c2) A2:Gree(v1,c1) A3:Repu(v3,c1) A4:Repu(v2,c1) A5:Repu(v3,c1)	100.0%	69	66.6
3	Preset 61	3×Frontier + 2×Repulsion	A1:Fron(v1,c1) A2:Fron(v2,c1) A3:Repu(v3,c1) A4:Repu(v2,c1) A5:Fron(v1,c2)	100.0%	71	68.4
4	Preset 1	2×Frontier + 1×Greedy + 2×Repulsion	A1:Repu(v2,c1) A2:Fron(v2,c2) A3:Fron(v1,c1) A4:Repu(v3,c2) A5:Gree(v2,c1)	100.0%	72	69.4
5	Preset 43	1×Frontier + 1×Greedy + 1×Repulsion + 1×Sector + 1×Smart Random	A1:Repu(v3,c2) A2:Gree(v2,c1) A3:Sect(v3,c1) A4:Smar(v1,c1) A5:Fron(v3,c2)	100.0%	72	70.2
6	Preset 40	1×Greedy + 1×Sector + 3×Smart Random	A1:Smar(v3,c1) A2:Sect(v3,c2) A3:Smar(v1,c1) A4:Gree(v2,c1) A5:Smar(v1,c1)	100.0%	72	70.4
7	Preset 4	3×Frontier + 1×Repulsion + 1×Sector	A1:Repu(v1,c1) A2:Sect(v1,c2) A3:Fron(v1,c2) A4:Fron(v1,c1) A5:Fron(v2,c1)	100.0%	74	72.4
8	Preset 100	1×Frontier + 1×Greedy + 2×Sector + 1×Smart Random	A1:Gree(v2,c2) A2:Fron(v3,c2) A3:Sect(v2,c1) A4:Smar(v2,c1) A5:Sect(v3,c2)	100.0%	74	72.4
9	Preset 84	2×Frontier + 2×Repulsion + 1×Sector	A1:Fron(v1,c2) A2:Repu(v1,c1) A3:Repu(v3,c2) A4:Sect(v3,c2) A5:Fron(v1,c2)	100.0%	75	71.6
10	Preset 76	2×Frontier + 1×Repulsion + 1×Sector + 1×Smart Random	A1:Repu(v2,c1) A2:Sect(v3,c2) A3:Smar(v2,c1) A4:Fron(v1,c1) A5:Fron(v2,c1)	100.0%	75	73

Figura 5.22: Mappa B — seed 42: classifica Top 10.

Seed 43

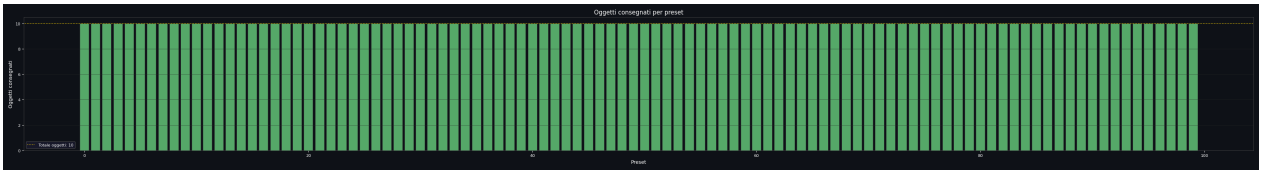

Figura 5.23: Mappa B — seed 43: oggetti consegnati per preset.

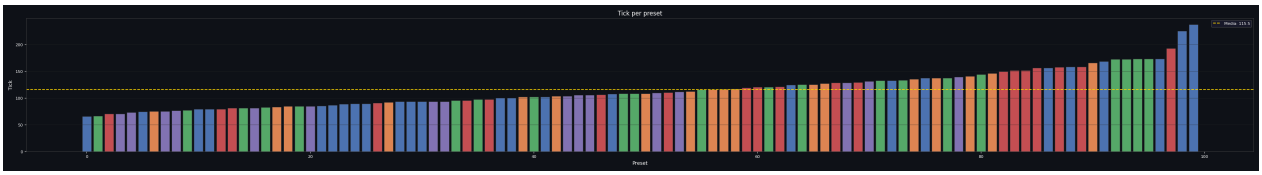

Figura 5.24: Mappa B — seed 43: tick di completamento per preset.

Top 10 preset

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 87	2×Frontier + 2×Repulsion + 1×Sector	A1:Fron(v2,c2) A2:Fron(v3,c1) A3:Repu(v1,c1) A4:Repu(v3,c1) A5:Sect(v3,c1)	100.0%	65	62.2
2	Preset 72	1×Frontier + 1×Greedy + 1×Repulsion + 2×Sector	A1:Sect(v2,c1) A2:Repu(v3,c1) A3:Gree(v2,c1) A4:Sect(v2,c2) A5:Fron(v3,c1)	100.0%	66	64
3	Preset 58	1×Frontier + 1×Greedy + 2×Repulsion + 1×Sector	A1:Fron(v1,c1) A2:Repu(v3,c2) A3:Gree(v1,c1) A4:Sect(v2,c2) A5:Repu(v2,c1)	100.0%	70	66.8
4	Preset 46	1×Frontier + 2×Sector + 2×Smart Random	A1:Smar(v3,c2) A2:Sect(v3,c1) A3:Smar(v2,c2) A4:Sect(v3,c1) A5:Fron(v2,c2)	100.0%	70	68.2
5	Preset 30	1×Frontier + 1×Repulsion + 3×Smart Random	A1:Repu(v3,c2) A2:Fron(v1,c1) A3:Smar(v1,c1) A4:Smar(v3,c1) A5:Smar(v2,c1)	100.0%	73	71.4
6	Preset 39	3×Frontier + 1×Sector + 1×Smart Random	A1:Fron(v1,c2) A2:Fron(v3,c2) A3:Sect(v3,c2) A4:Fron(v3,c2) A5:Smar(v1,c2)	100.0%	74	72.4
7	Preset 57	1×Frontier + 3×Greedy + 1×Repulsion	A1:Repu(v3,c1) A2:Gree(v3,c1) A3:Fron(v3,c1) A4:Gree(v3,c1) A5:Gree(v1,c2)	100.0%	75	72.6
8	Preset 16	2×Repulsion + 1×Sector + 2×Smart Random	A1:Sect(v2,c1) A2:Smar(v1,c2) A3:Repu(v2,c1) A4:Smar(v3,c1) A5:Repu(v3,c2)	100.0%	75	73
9	Preset 29	2×Frontier + 3×Smart Random	A1:Smar(v3,c1) A2:Smar(v1,c2) A3:Smar(v2,c1) A4:Fron(v2,c2) A5:Fron(v3,c1)	100.0%	76	74.4
10	Preset 89	1×Repulsion + 4×Sector	A1:Repu(v2,c1) A2:Sect(v2,c1) A3:Sect(v3,c2) A4:Sect(v2,c2) A5:Sect(v3,c2)	100.0%	77	74.4

Figura 5.25: Mappa B — seed 43: classifica Top 10.

Seed 44

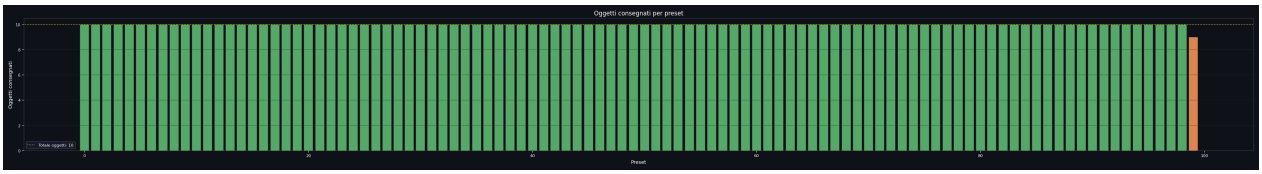

Figura 5.26: Mappa B — seed 44: oggetti consegnati per preset.

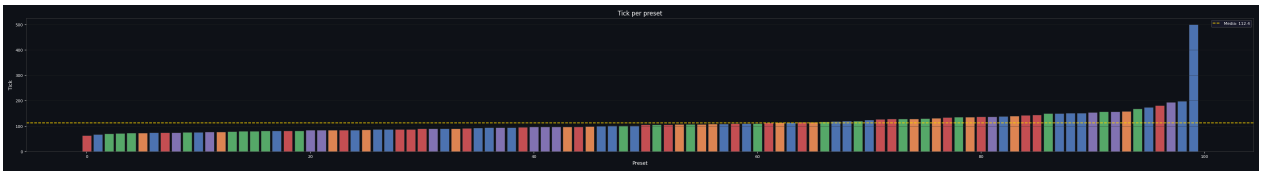

Figura 5.27: Mappa B — seed 44: tick di completamento per preset.

🏆 **Top 10 preset**

Pos.	Preset	Team	Configurazione	Completion	Tick	Energia media
1	Preset 100	1×Greedy + 2×Repulsion + 1×Sector + 1×Smart Random	A1:Repu(v1,c1) A2:Sect(v3,c1) A3:Repu(v2,c2) A4:Smar(v3,c2) A5:Gree(v1,c1)	100.0%	63	61.2
2	Preset 63	4×Frontier + 1×Sector	A1:Fron(v1,c1) A2:Fron(v3,c2) A3:Fron(v2,c1) A4:Fron(v3,c2) A5:Sect(v3,c2)	100.0%	66	63.8
3	Preset 82	1×Frontier + 1×Greedy + 2×Sector + 1×Smart Random	A1:Smar(v3,c2) A2:Gree(v3,c1) A3:Sect(v2,c1) A4:Sect(v2,c2) A5:Fron(v1,c1)	100.0%	70	68.4
4	Preset 92	1×Frontier + 1×Greedy + 1×Repulsion + 2×Sector	A1:Repu(v2,c1) A2:Sect(v2,c2) A3:Fron(v2,c2) A4:Sect(v3,c2) A5:Gree(v3,c1)	100.0%	71	69
5	Preset 73	1×Frontier + 1×Greedy + 1×Repulsion + 1×Sector + 1×Smart Random	A1:Sect(v2,c1) A2:Fron(v2,c2) A3:Smar(v1,c2) A4:Repu(v3,c1) A5:Gree(v2,c1)	100.0%	72	69.8
6	Preset 43	1×Frontier + 2×Greedy + 1×Repulsion + 1×Sector	A1:Gree(v1,c2) A2:Fron(v1,c2) A3:Repu(v3,c1) A4:Gree(v1,c2) A5:Sect(v3,c2)	100.0%	72	70.2
7	Preset 8	3×Frontier + 1×Repulsion + 1×Sector	A1:Repu(v1,c2) A2:Fron(v3,c2) A3:Fron(v2,c2) A4:Sect(v3,c1) A5:Fron(v2,c1)	100.0%	74	72
8	Preset 28	1×Frontier + 3×Repulsion + 1×Smart Random	A1:Repu(v2,c1) A2:Repu(v3,c2) A3:Smar(v2,c1) A4:Fron(v1,c1) A5:Repu(v1,c2)	100.0%	74	72.4
9	Preset 75	1×Frontier + 1×Repulsion + 1×Sector + 2×Smart Random	A1:Sect(v2,c2) A2:Smar(v3,c1) A3:Repu(v2,c1) A4:Smar(v1,c1) A5:Fron(v3,c2)	100.0%	74	72.6
10	Preset 95	1×Frontier + 1×Repulsion + 2×Sector + 1×Smart Random	A1:Sect(v1,c1) A2:Fron(v1,c1) A3:Smar(v2,c1) A4:Repu(v3,c2) A5:Sect(v3,c2)	100.0%	75	72.4

Figura 5.28: Mappa B — seed 44: classifica Top 10.

5.3 Considerazioni sui risultati

5.3.1 Confronto tra le due mappe

La Mappa B risulta sistematicamente più semplice della Mappa A su tutte le metriche. Nel test esaustivo il miglior preset completa in 57 tick (contro i 77 della Mappa A), con un tick medio di completamento di 105,7 contro 162,9 (−35 %) e un consumo energetico medio di 103,3 contro 161,1 (−36 %). Il tasso di completamento sale dal 96,4 % al 99,6 %. Questa differenza è attribuibile alla topologia della mappa: una pianta più aperta riduce i cammini medi agente-oggetto e agente-magazzino, abbassando sia i tick sia l'energia necessari.

5.3.2 Impatto della strategia dominante

Analizzando i 32.768 preset completati per strategia dominante (la strategia che compare più volte nel team di 5 agenti), emerge una chiara differenza tra le due mappe:

- **Mappa A**: le quattro strategie ottengono tick medi molto simili (Repulsion 158,7, Greedy 161,5, Sector 163,0, Frontier 168,5), con Repulsion leggermente avvantaggiata. Nella Top 10, Sector domina con 17 assegnazioni su 50, seguito da Greedy (12) e Repulsion (11).

- **Mappa B**: Repulsion emerge nettamente come strategia migliore (101,5 tick medi, −9 % rispetto a Frontier).W Nella Top 10 compare 23 volte su 50, contro 13 di Sector e 10 di Frontier.

Nota sulla posizione sistemática di Sector nei team Top 10: Sector appare sistematicamente inserito in 4ª posizione (indice agente 3) nei team Top 10 di entrambe le mappe. Questo fenomeno non è casuale: la strategia Sector divide la griglia in bande orizzontali assegnate agli agenti in base a `agent_id` mod N. Agenti con ID inferiore occupano bande più alte e procedono verso il basso in modo sistematico. Posizionare Sector in 4ª posizione minimizza la sovrapposizione con gli altri agenti: se Sector avesse ID inferiore, la sua banda occuperrebbe spazio sin dall'inizio della mappa, potenzialmente entrando in conflitto con le traiettorie delle altre strategie. Questa configurazione emerge naturalmente dall'ottimizzazione, riflettendo il trade-off tra copertura sistematica di Sector e flessibilità delle altre strategie nei team misti.

5.3.3 Team eterogenei vs omogenei

I team omogenei (tutti e 5 gli agenti con la stessa strategia) rappresentano solo 128 dei 32.768 preset per ciascuna mappa. I risultati sono eloquenti:

- **Mappa A**: team omogenei completano nel 96,9 % dei casi con una media di 172,5 tick; team eterogenei completano nel 96,4 % con 162,9 tick e ottengono il record assoluto di 77 tick. Nessun team omogeneo scende sotto i 99 tick.
- **Mappa B**: team omogenei completano nel 100 % dei casi con 102,3 tick medi, ma il miglior risultato è 64 tick; i team eterogenei raggiungono 57 tick.

In entrambe le mappe il **miglior preset assoluto è sempre eterogeneo**. Un team misto beneficia della complementarità tra le strategie: Sector fornisce copertura sistematica, Repulsion garantisce dispersione, Greedy ottimizza i percorsi verso i magazzini e Frontier esplora le zone residue.

5.3.4 Impatto del raggio di visibilità

Raggruppando i preset completati per raggio di visibilità medio del team ($\bar{v}_r$), si osserva un trend diverso nelle due mappe:

- **Mappa A**: i tick medi calano da 175,1 ($\bar{v}_r = 2{,}0$) a 152,6 ($\bar{v}_r = 2{,}8$), ma risalgono a 162,9 per $\bar{v}_r = 3{,}0$. Il raggio ottimale si colloca intorno a $\bar{v}_r \approx 2{,}6$–$2{,}8$: un campo visivo troppo ampio aumenta il costo computazionale del ray casting senza aggiungere informazione utile in corridoi stretti.
- **Mappa B**: i tick calano monotonicamente da 116,1 ($\bar{v}_r = 2{,}0$) a 96,7 ($\bar{v}_r = 3{,}0$). Su una mappa aperta un raggio maggiore è sempre vantaggioso perché incrementa direttamente l'area scansionata ad ogni tick.

Nonostante queste differenze, i Top 10 di entrambe le mappe mantengono un mix di raggi 2 e 3, suggerendo che la diversità percettiva nel team contribuisce alla robustezza.

5.3.5 Robustezza ai seed casuali

Il test randomizzato con 3 seed indipendenti consente di stimare la sensibilità al rumore stocastico. I risultati sono stabili:

- **Mappa A**: i tick medi oscillano tra 155,5 e 163,1 nei tre seed, con una variazione massima del ±2,4 % rispetto alla media. Il tasso di completamento è costante (96–97 %).
- **Mappa B**: i tick oscillano tra 108,5 e 115,5 (±3,1 %). Il completamento è quasi perfetto (99–100 %) su tutti e tre i seed.

La bassa varianza inter-seed conferma che le prestazioni osservate dipendono prevalentemente dalla configurazione del team e dalla topologia della mappa, non dalla specifica distribuzione casuale degli oggetti.

5.3.6 Riepilogo

1. La topologia della mappa è il fattore più influente: mappe aperte (B) sono risolvibili il 35 % più velocemente di mappe strutturate (A).
2. I team eterogenei battono sistematicamente quelli omogenei nei tempi migliori, sfruttando la complementarità tra strategie diverse.
3. Repulsion è la strategia più performante su mappe aperte; Sector e Greedy risultano competitivi su mappe con corridoi.
4. Un raggio di visibilità elevato è sempre vantaggioso su mappe aperte, mentre su mappe strutturate esiste un punto di rendimento decrescente intorno a $\bar{v}_r \approx 2{,}6$–$2{,}8$.
5. Il sistema mostra buona robustezza statistica, con variazioni contenute al variare del seed casuale.

5.3.7 Riepilogo delle euristiche utilizzate

Il sistema impiega un insieme composito di euristiche, ciascuna con un ruolo specifico nel coordinamento e nella decisione dei team multi-agente:

Ray-casting (Bresenham) Calcola la visione dell'agente entro raggio v_r considerando l'occlusione da muri. Base della percezione distribuita.

Coverage targets Fornisce tre livelli di priorità per l'esplorazione: celle inesplorate, celle stale (non visitate di recente), pattugliamento. Guida tutte le strategie verso aree di maggior interesse.

Guadagno informativo (*info_gain*) Per una cella candidata t, conta le celle non ancora viste che l'agente osserverebbe da quella posizione:

$$\text{info_gain}(t) = \left| \{(r,c) : |r - t_r| + |c - t_c| \leq v_r \ \wedge \ (r,c) \notin \text{seen_cells}\} \right|$$

Usato da Smart Random Walk e Greedy per favorire mosse verso aree dense di celle inesplorate.

Scelta dell'ingresso di consegna (Costo dinamico) Per un ingresso e combina distanza, congestione e prenotazioni:

$$C(e) \ = \ d(\text{agent}, e) \ + \ 4{,}5 \cdot Q_{\text{local}} \ + \ 1{,}5 \cdot R_{\text{reserved}} \ + \ \varepsilon_{\text{tie}}$$

dove Q_{local} sono gli agenti noti entro raggio 2 e R_{reserved} è il numero di prenotazioni attive. Scelta rideterminata a ogni tick, con hysteresis del 20% per evitare oscillazioni.

Separazione (Repulsion) Massimizza la distanza media dagli agenti noti:

$$\text{score}(t) = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} d(t,a) - d(\text{agent}, t)$$

Induce dispersione naturale senza comunicazione esplicita; efficace su mappe aperte.

Frontiere (*frontier-based*) Identifica celle visitate che confinano con celle non visitate. Classico approccio all'esplorazione sistematica con A* per il raggiungimento.

Costo composito (Greedy) Minimizza il costo composito di esplorazione:

$$\text{cost}(t) = d(\text{agent}, t) + d(t, \text{wh}_{\text{nearest}}) - \text{info_gain}(t)$$

Bilancia distanza, prossimità al magazzino più vicino, e guadagno informativo. Produce bias verso zone inesplorate ma raggiungibili efficientemente.

Pattern a banda (Sector) Divide la griglia in bande orizzontali assegnate per `agent_id` mod N. Garantisce copertura sistematica e uniforme senza comunicazione di coordinamento.

Punteggio multi-criterio (Smart Random Walk) Calcola il punteggio come somma pesata di cinque criteri:

$$\text{score}(n) = w_{\text{info}} \cdot \text{info_gain}(n) + w_{\text{stale}} \cdot \text{age}(n) - w_{\text{revisit}} \cdot \text{penalty}(n) + w_{\text{sep}} \cdot \text{separation}(n) + \mathcal{U}(0,\ 0{,}3)$$

con pesi: $w_{\text{info}} = 1{,}2$, $w_{\text{stale}} = 0{,}8$, $w_{\text{sep}} = 0{,}5$, $w_{\text{revisit}} = 1{,}0$. Softmax per selezione probabilistica.

Costo-beneficio con hysteresis Applica un 20% di soglia di miglioramento prima di cambiare destinazione di consegna (lock di 8 tick). Riduce oscillazioni e overhead comunicativo.

Pathfinding A* Ricerca del percorso più breve verso target con euristica Manhattan. Le celle occupate da altri agenti vengono passate come set di celle bloccate per evitare collisioni. Supporta modalità specifica per transito attraverso il magazzino durante consegna.

Finestra di validità temporale Le posizioni degli agenti noti sono valide per 5 tick. Informazioni più vecchie vengono scartate durante la pianificazione locale per evitare collisioni con agenti spostati nel frattempo. Permette agli agenti di gestire un modello parziale del sistema pur mantenendo sicurezza.

Stale detection e pulizia Celle non visitate entro $\max(8, N/2)$ tick vengono riclassificate come *stale* nei coverage targets. Gli oggetti rimossi dalla visibilità diretta vengono eliminati da `known_objects` per evitare ricerche fantasma. Mantiene coerenza tra percezione e memoria locale.

Gossip protocol distribuito Ad ogni comunicazione, gli agenti scambiano bidirezionalmente: mappa locale, oggetti noti, posizioni di agenti conosciuti (merge con timestamp), prenotazioni di consegna. Non esiste canale broadcast globale; la conoscenza si propaga per contatto diretto, creando un effetto di diffusione graduale che favorisce robustezza e decentralizzazione.

Priorità deterministiche nelle collisioni In caso di conflitto per la stessa cella: priorità a chi trasporta un oggetto; a parità di stato, vince l'agente con ID inferiore. Garantisce risoluzione senza deadlock e comportamento prevedibile nel debugging. Gli agenti bloccati restano fermi e riprovano al prossimo tick.

Protezione delle porte magazzino Celle `ENTRANCE` e `EXIT` sono automaticamente protette come celle bloccate durante la pianificazione A*. Impedisce che vengano usate come scorciatoie o transiti intermedi, riducendo congestione e garantendo che siano raggiunte solo come destinazione finale di un percorso.